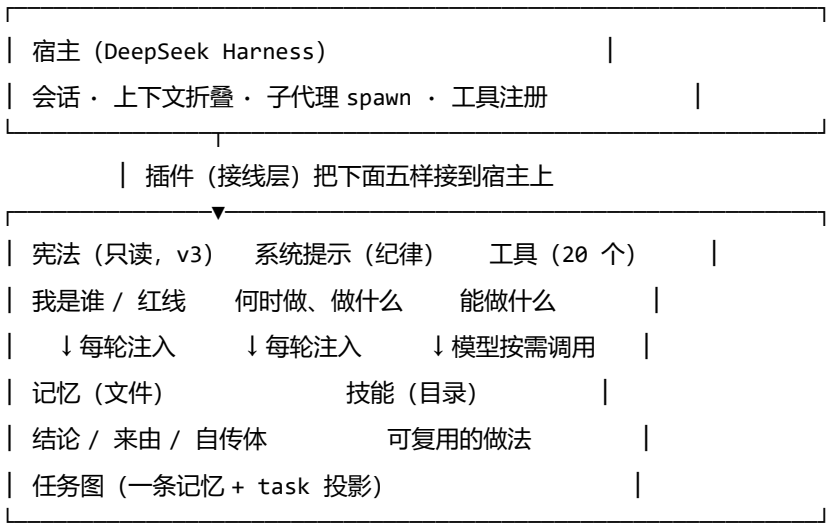
Mnemos · 架构总览（v1.7.0）

> ***这份文档只回答三件事：它由哪些部件组成、任务怎么跑通、三种召回模式怎么分工。***
>
> ***每个机制的"保证什么 / 边界在哪"见《机制清单》；为什么这么设计见《设计者视角》；怎么装怎么配见 INSTALL.md。***

>

> ***版本事实以 00-事实表-v1.7.0.md 为准；本文不重复所有数字。***

一、五个部件，加把它们接起来的插件



部件	是什么	怎么进到模型眼前	一句话职责
宪法	一份只读文件，当前 v3（认知体）	每轮注入提示最前（最高优先级）	它是谁、红线在哪
系统提示	一段固定的纪律文本（25 条 / 7 节）	每轮注入	什么时候该做什么
工具	20 个可调用接口	由插件注册给宿主	它能做什么、不能做什么
记忆	本地 JSON 文件	按需检索（两段式：先摘要后详情）	结论与来由放哪

技能	一个目录	目录清单注入 + 按需加载全文	可复用的做法放哪
任务图	一条记忆记录 + task 投影	由工具打开/更新/关闭；按 graphId 追查	任务寻址、工作记忆、档案、来源锚点
插件	接线层，不算第六个部件	——	把上面几样接到宿主上：写入安全、注入、心跳、闸门

关键区分：宪法、系统提示是"每轮都在场"的；记忆、技能、任务图是"按需取用/寻址"的。前者的体积必须受控，后者可以很多。

二、身份每轮怎么回到它眼前（自传体注入）

"换掉模型，它还是同一个它"不靠模型自己记住，而是每次请求都把"我是谁"重新放到它眼前。

```
永久层里的自传体（identity / decision / experience / event / other）
        │
    │ 每次组装提示时重新计算（系统提示变量，不是写死文本）
      ▼
【自传体 · 自我历史（紧凑版，按需召回详情）】
[身份] ……
[决策] ……
[经验] ……
        │
    └─ 装不下的会明说：另有 N 条未进本段，需要时用召回取
```

当前固定注入：

```
identity core（全部） + 最近 3 条 decision + 最多 5 条 experience
= 12 条 / 约 1167 字（4 身份 / 3 决策 / 5 经验）
```

要点	具体做法
存在哪	自传体落在永久层：不衰减、不归档，后台维护任务清不走
怎么进去	作为系统提示变量注入，每次组装提示时重新计算
注入什么	身份 core + 最近 3 决策 + 最多 5 经验；不是"全部自传体"
详情在哪	注入的是摘要；完整记录用 mnemos_recall + mnemos_open 取
边界	固定注入有预算；装不下的会明说"另有 N 条未进本段"

一句话：身份不是"被模型记住的"，是"每一轮被重新放到它眼前的"。

三、三种召回模式（v1.7 的核心变化）

召回不再只有"关键词 + 标签"一条路，而是三层：

① 固定注入（自传体）

每轮自动在场：身份、最近判断、关键经验

② 任务激活（模型主动召回）

`mnemos_recall / mnemos_open`

模型在工作里觉得缺背景时主动查

③ 联想激活（机械匹配候选，实验性）

工具执行后附注候选 `id + 摘要片段`

不新增 `user message`、不自动 `open`、不保证相关

① 固定注入

- 由插件在每轮组装系统提示时完成；
- 只负责"我是谁、最近判断、关键经验"；
- 当前 12 条 / 1167 字；
- 不是任务状态库，不是会话记录仓库。

② 任务激活

模型觉得缺背景

→ `mnemos_recall`（关键词/标签/类目/taskId）

→ 返回摘要 + `id`

→ 需要细节时 `mnemos_open(id)` 取全文

这是主通道。可靠性取决于模型是否意识到"该查"；这也是系统提示反复强调"开始任务前先 recall"的原因。

③ 联想激活

工具执行成功

→ 插件用本轮可见文本做机械匹配

→ 候选 `top 3` 追加到 `tool result` 附注

→ 只给 `id + 摘要片段`，让模型自己决定要不要 `open`

当前定位：

- 实验性/早期质量；
- 候选可能相关，也可能无关；
- 不是第二套召回承诺，不保证命中率；
- 不自动把全文塞进上下文；
- 桌面 profile 开启，web 未开；配置项尚未正式写入 Config schema。

> ***文档必须诚实：联想激活是"搭车提供线索"，不是"已经验证的提升"。***

四、任务图：底层机制，不是协作小节

任务图是设计上最简单、功能上最广的底层机制：

一条记忆记录 + task 投影
`tags: task-graph / open|closed / task:<taskId>`

它同时承担五件事：

功能	具体表现
任务寻址空间	taskId / graphId；mnemos_task_open/update/close/done/cite
部分工作记忆	自己的事：goal / done / uses / links；协作：assignments / handoff / 未完成分工
档案与追查入口	摘要命中后按 graphId 打开图，核对 uses / links / origin
事件/决策来源锚点	自传体 origin={kind:'task',taskId,graphId}；task-result 带 graphId；close event 带 graphId
归档闸门	open task 下的短时记录不被自动归档；收尾后才放行

关键边界：

- 任务图不是待办清单，不是工作流引擎，不复制正文；
- 未完成步骤用宿主 todo，图里不记待办；
- 单写者：只有创建图的会话能 update/close；
- close 只写/补收尾 event，成员转 cold，不合并成员；
- 跨会话续接开新图并用 links 链接旧图，不改旧图。

> ***一句话：任务图是任务寻址空间 + 部分工作记忆 + 档案/追查入口 + 事件/决策来源锚点。***

五、五条主要路径

① 写一条记忆

模型判断"值得留" → remember → 插件补齐 sessionId / 层级 / 来源

→ 写前重读磁盘 + 合并 + 加锁 → 落盘 → 回执（可能带"冲突旧记忆"提示）

写入是显式动作；写之前要先看磁盘上现在是什么。内容 tag 可双语同串（如 learner 学习）；summary 用中文一句话，关键词处可附英文对照（如 任务图（task graph）），不整句翻译。

② 取一条记忆

模型觉得缺背景 → recall（关键词/标签/类目/taskId） → 排序

→ 返回摘要 + id → 需要细节时 open(id) → 取全文

两段式：先用便宜的方式判断"哪些相关"，再为真正需要的那几条付全文成本。

记忆文件按 profile 定位，与"你在哪个目录开会话"无关。

③ 上下文折叠时

宿主按位置折叠最旧一段 → 插件收到折叠事件

→ 从被遮蔽区域抽三类硬事实（工具失败 / 用户原话 / 负面结论）

→ 有硬事实才存一条事实卡（不存 4K 原文，7 天后自动清理）

→ 往会话里注入提醒：最多 3 条负面结论 / 3 条工具失败 + 记忆 id

折叠是宿主的决定，插件拦不住，只能抢救；注入只带指针、不搬正文。

④ 技能自己变好

一段"失败过又成功"的工作流结束 → 计数触发（零成本，不调模型）

→ 提议新技能：反例必须能机械追溯到真实失败 + 正例

→ 独立验证者（看不到修改理由）→ 回归门 → 写回（旧版留档）

→ 或者：拒绝 / 什么都不改（这也是有效结论）

验证与生成分开；改的是已经被用过的东西，所以默认留档。

⑤ 多 agent 协作

主模型 open 任务图（一条记忆）→ 派活，只给图的 id

→ 子代理干活，把结论写成短时记忆（带 task:/assign: 标签）并回报 handoff id

→ 主模型把回执填回图（唯一写者）→ close：写收尾 event，成员转 cold

→ 没回执的分工如实标未完成；超期未收尾的图由后台强制归档

协作状态在记忆层，不在 prompt 里；未收尾前这批记录不许被归档。

六、一轮正常任务的生命周期

开始：固定注入已就位；模型按需 `recall` 相关记忆

↓

干活：调用工具；外部信息先隔离验证；大块输出先卸载再摘要

↓

结束：写结论；任务图 `close`；本会话短时记忆归档

↓

以后：下一个会话用 `recall / graphId` 找回结论、来由与是否被推翻

七、部件之间的"不能做什么"

谁	不能做什么
模型	不能改宪法；不能把外部信息直接当事实；不能绕过工具直接写记忆文件
技能	不能改宪法；不能在没有独立验证的情况下改自己
记忆	不会自动改代码、改配置；除有界回收外不会自动删除自己（只降权，不删）
插件	不碰用户数据文件以外的东西；删除只走显式工具或有界回收（隔离验证记录）
宿主	折叠是它的权力——插件只能抢救，不能阻止

八、这张地图不包含什么

- 不列每个机制的保证与边界 →《机制清单》；
- 不解释为什么这样设计、被否掉的替代方案 →《设计者视角》；
- 不写代码结构与数据结构 →《模块与功能实现》；
- 不写安装与配置 → INSTALL.md。